树形借用模型

Neven Villani
第一作者
格勒诺布尔大学

Johannes Hostert
第一作者
苏黎世联邦理工大学

Derek Dreyer
作者
马克斯-普朗克
软件系统研究所

Ralf Jung
作者
苏黎世联邦理工大学

Chuigda Whitegive
翻译
Doki Doki λ Club!

Claude
校对
Anthropic

⚠ 注意：本文为早期草稿，内容不完且有措误，且排版质量差。

⚠ Note: this is an early draft. It's known to be incomplet and incorrekt, and it has lots of bad fomatting.

译者前言

本文是文章 Tree borrows 的中文翻译，部分字句有所改动。原文以 CC-BY 4.0 协议发布。术语 *(terminology)* 在正文中第一次出现的地方以仿宋体（中文）或 *Italic (English)* 呈现，如果某个术语难以辨认，则总是会以仿宋体呈现。如遇翻译或排版质量问题，请在 https://github.com/club-doki7/naql/issues 向译者报告。

栈式借用模型 *(Stacked Borrows)* 的概念早在 2018 年就已被提出，并于 2020 年作为学术论文发表并应用于 `miri`。而树形借用模型 *(Tree Borrows)* 的思想也在 2025 年得到了形式化，并且也已经在 `miri` 中得到了实际运用。遗憾的是，这些工作在中文 Rust 社区并未得到应有的广泛认知。许多人将 `unsafe` 视作可以为所欲为的免死金牌，在使用 `unsafe` 时忽视别名规则。译者翻译本文的主要动机之一便是纠正这一常见误解。

摘要

Rust 程序设计语言因其基于所有权的类型系统而闻名，这一类型系统为内存安全和无数据竞争提供了强有力的保证。然而，Rust 也提供了非安全 *(unsafe)* 代码这一"逃生舱门"：在 `unsafe` 的领域中并不自动保障安全性，一切全凭程序设计者自觉。这便引出一种矛盾：一方面，编译器希望能尽可能利用类型系统的强力保证——特别是与指针别名相关的保证——以实现强大的过程内优化；另一方面，这些优化易因"行为不良"的非安全代码而失效。因此，要确保优化的正确性，就必须清晰地定义什么样的非安全代码算是"行为不良"。之前的工作——栈式借用模型——为此定义了一系列规则。然而，栈式借用模型排除了几个在实际的 Rust 非安全代码中常见的模式，且并未考虑到 Rust 借用检查器近期引入的一些高级特性。

为解决这些问题，我们提出了树形借用模型。顾名思义，树形借用模型将栈式借用模型的核心替换为树形结构。这就克服了先前所述的限制：我们评估了 30 000 个最常用的 Rust crate，树形借用模型拒绝的测试用例相比栈式借用模型减少了 54%。此外我们还（在 Rocq 中）证明了新模型在保留栈式借用模型的大部分优化的同时，还让一些重要的新优化成为了可能——特别是读-读重排序 *(read-read reordering)*。

计算机科学概念：• **计算理论 → 操作语义**

额外关键词：Rust，操作语义，编译器优化，Miri，Rocq

ACM 引用格式： Neven Villani, Johannes Hostert, Derek Dreyer, and Ralf Jung. 2025. Tree Borrows. Proc. ACM Program. Lang. 9, PLDI, Article 188 (June 2025), 24 pages. https://doi.org/10.1145/373559

1. 简介

类型系统已被证明能以可组合的方式高效地排除整类错误。Rust 正是最近的成功案例，它使用仿射 *(affine)*类型（“基于所有权的”类型）和生存期 *(lifetime,* 区域类型 *region types* 的变体*)* 来静态地确保底层系统应用程序的类型安全、内存安全和无数据竞争，而无需垃圾回收器。

然而，类型系统的好处不只是确保程序不会出错：类型系统还能提高程序的运行速度。事实上，Rust 编译器从一开始就将其类型系统用于优化，而不只是确保安全。特别地，Rust 的类型中编码了强大的别名信息：可变不共享，共享不可变。别名分析是现代优化器的核心支柱，因此 Rust 编译器的开发者希望将无处不在的类型信息用于别名分析也就不足为奇了。考虑这个具体的例子：

```rust
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
    *x = 13;
    *y = 20;
    *x // 返回 13
}
```

例 1

函数 `write_both` 接受两个可变引用作为实参，而根据 Rust 的类型系统规则，这两个引用不可能互为别名。由此可知，该函数必须始终返回 `13`。因此最后一行中从 `*x` 读取数据的操作可被移除，返回值可被硬编码为 `13`，而这些变换不会影响程序的行为。虽说这个例子是人为设计的，但它展示了一类强大的过程内优化：无论内存是在程序中何处分配的，编译器总是可以作这种变换。

1.1. 非安全代码带给别名优化的挑战

不幸的是，Rust 所面临的现实比这复杂得多。Rust 的类型系统所提供的别名保证有时过于强大，使得程序设计者无法实现某些抽象（例如，依赖于共享状态变更的抽象）或是达到预期的性能。因此，程序设计者有时需要使用非安全代码——尤其是原始指针 *(raw pointer)*——来规避类型系统的限制，而原始指针的别名是不受追踪的。Rust 程序设计者会将非安全代码封装在安全的抽象层中以保持类型安全推理的组合性。例如，标准库中的 `Vec` 类型提供了大小可变的数组。`Vec` 类型内部是用非安全代码实现的，但它提供给用户的公共 API 却是安全的。这意味着数百万行代码可以使用 `Vec` 而无需担心类型安全问题，只有实现 `Vec` 的那几千行代码需要额外的审查。

非安全代码的问题在于，用它们可以进行如下操作：

```rust
fn main() {
    let mut x = 42;
    let ptr = &mut x as *mut i32;
    let val = unsafe { write_both(&mut *ptr, &mut *ptr) };
    println!("{val}");
}
```

例 2

以上代码先是创建了指向可变局部变量 `x` 的原始指针 `ptr`，然后用 `unsafe` 代码块，两次将同一个原始指针转型为可变引用，这样就有了两个互为别名的可变引用。因此若不带优化地编译，则这段程序会输出 `20`，但若按照先前所述的方法优化 `write_both`，则程序会输出 `13`。坏！优化不应该改变程序的行为。

Rust 编译器的开发者确是希望支持别名优化的，这就需要某种方法来“排除”上面这样的反例。Rust 程序设计者早已习惯了这一点：使用非安全特性不意味着他们可以为所欲为，编写非安全代码的开发者必须格外注意特殊规则。例如，调用 `get_unchecked` 函数以在不进行边界检查的前提下访问数组时，非安全代码的作者必须确保索引在数组边界之内。不符合这些要求的非安全代码包含未定义行为 *(Undefined Behavior, UB)*，因为由此产生的程序可能会以任意方式出现异常行为。因此，非安全代码的作者有义务确保他们的代码没有未定义行为；而编译器会反过来信任程序设计者，并假定它所编译的代码中没有未定义行为。

要保留例 1 中基于引用的优化，我们需要找到一种方法，使得例 2 中的程序具有未定义行为。但这说来容易做来难。例如，以下完全安全的 Rust 代码片段看似也会导致可变引用别名问题：

```
let mut x = 42;
let ref1 = &mut x;
let ref2 = &mut *ref1;
*ref2 = 12;
println!("{ref1}"); // 输出 12
```

例 3

ref1 和 ref2 都指向 x。Rust 若是断然不许可变引用互为别名，又怎能允许这种状况呢？因为这是一种重借用 *(reborrowing)*：ref2 派生自 *(derived from)* ref1，故编译器允许它们同时存在。然而当 ref2 存活时，ref1 便不能使用；而当 ref1 再被使用时，即是 ref2 生存期的终点。若在例 3 的结尾添上一行 `println!("{ref2}")`，就违反了这规则，程序便编译不成了。

因此，摆在我们面前的挑战便是：如何定义“何谓未定义行为”，使得形如例 2 的反例被拒绝，但所有安全 Rust 代码都被接受。事实上，我们还要更进一步，确保足够多“合理”的非安全代码也被接受：必须让程序设计者能以适度的努力编写出正确的非安全代码，非安全 Rust 才真正称得上可用。

1.2. 先前的工作：栈式借用模型

我们不是第一批勇攀高峰的人：栈式借用模型 [1] 有着完全相同的目标。栈式借用模型注意到，在例 3 这样的场景中，当前有效的指针可被记录在一个栈中：x 在栈底，ref1 在它上面，而 ref2 在栈顶。栈模型是对这一事实的建模：引用的所有使用必须是良嵌套的 *(well nested)*。当 ref1 被使用时，ref2 就会失效。不幸的是，简单的栈模型有三大问题。

栈式借用模型的第一个问题在于它会阻止一些基本的优化，例如重排序相邻的读取操作。原因很简单：在例 3 中，若先读取 ref2 再读取 ref1，那么一切相安无事。ref2 是“嵌套于” ref1 中的，所以所有访问都是允许的。然而，若重排序这两个操作，就成了先读取 ref1 再读取 ref2。这在栈式借用模型下是未定义行为，因为这违反了良嵌套规则。读取重排序是一项至关重要的优化，因此这是一个显著的弊端[1]。

栈式借用模型的第二个问题在于它将引用限定在一个静态范围内：引用被创建之后，引用的类型决定了它所能访问的内存范围，无论是引用本身还是由它所派生出的指针，均不能访问这一范围外的内存。而在实践中，这往往并非程序设计者所期望的行为。例如，有 C 语言经验的程序设计者有时会创建指向数组首元素的引用，将其转成原始指针，然后用指针访问整个数组。栈式借用模型会拒绝这种操作，因为从指向数组首元素的引用派生出的原始指针只能访问这一个元素。程序设计者的心智和实际模型之间的这一差异导致现实代码中频繁出现未定义行为 [2–6]，因此，一个更宽松的替代选项呼之欲出。

第三个问题则相当技术性，它涉及一个与栈式借用模型同期开发的 Rust 借用检查器特性：两阶段借用 *(two-phase borrows)*。这一特性弱化了对某些可变引用的要求：它们不必在创建之后马上就是唯一的；相反，它们在被创建时处于一个保留 *(reserved)* 阶段，这一阶段允许只读别名。只在可变引用第一次被写入前、被激活 *(activate / activation)* 时，唯一性才被强制执行。

原本的栈式借用模型完全不支持两阶段借用。Rust 程序设计者用栈式借用模型来检查代码中的未定义行为，而栈式借用模型的实现只对两阶段借用提供了简陋的支持：在检查含有两阶段借用的代码时，实现基本上是把两阶段借用当作原始指针处理的。这一简陋支持会产生一些非常反直觉的效果，时常令 Rust 程序设计者感到困惑，如下例所示：

[1]除了整个换掉栈式借用模型之外，这一缺陷还可通过为优化所操作的中间表示形式选用不同的模型来克服。但这仍然需要开发一个新模型，而因为该模型是栈式借用模型的细化，其必然受到更多约束——即中间表示模型具有的未定义行为只能比表层模型更少。为此，我们决定完全替换掉栈式借用模型。

```rust
fn write(x: &mut i32) { *x = 10; }

let x = &mut 0;
let y = x as *mut i32; // 从 x 中派生原始指针
write(x);              // 写入 x
unsafe { *y = 15 };    // 使用原始指针
```

例 4

栈式借用模型接受以上程序。然而，若将 `write` 调用替换为其函数体，则栈式借用模型会拒绝该程序[2]。造成这一区别的原因在于 `write(x)` 隐式地展开成了 `write(&twophase x)`，这里 `&twophase` 是一个伪语法，表示受两阶段借用规则约束的隐式可变引用。而栈式借用模型会将两阶段借用当作原始指针处理。这一差异不但会让程序设计者感到困惑，将隐式创建的可变引用视作指针也会限制优化潜力。

1.3. 我们的贡献：树形借用模型

在本文中，我们展示了一种能够克服以上局限的新别名模型，我们称之为**树形借用模型**：

- 要支持两阶段借用，就需要使用树代替栈来记录引用间的关系。栈结构已能确保每个引用都有唯一的父级：创建它的引用 / 局部变量。但与此同时，栈结构还强加了一个额外的约束：每个引用在同一时刻至多只能有一个子级是有效的。而这在处理两阶段借用时就力不从心了：处于保留阶段的可变引用可与多个共享引用和平共处。因而这些引用必须被作为同一个父级引用的独立子级记录，而当可变引用被激活时，其同级引用就不能再使用了。这样一来，使用树形结构也就顺理成章。
- 为支持读取重排序，树上的每个节点都记录一个状态机，该状态机表示由该节点表示的引用的当前权限。当该引用所指向的内存经由该引用或其他引用读写时，权限会相应地改变。这与栈式借用模型截然相反：栈式借用模型中，引用的权限在创建时便已固定，直至失效都不会改变。
- 最后，栈式借用模型的静态引用范围也被改为动态引用范围，引用声称唯一性（或者不变性）的内存区域不再有固定的界限。区域现在由引用的使用方式决定。这很大程度上依赖于树形结构：树形结构可以跟踪多个同级引用，而无需预先决定哪个引用对于内存的哪一部分是唯一的。

为了实证树形借用模型的有效性，我们在中层中间表示形式解释器 *(Mid-level Intermediate Representation Interpreter, Miri)* 中实现了这一模型——Miri 也曾被用于评估栈式借用模型。我们在下载数最多的 30 000 个 Rust 库上开展了实验，确认树形借用模型所拒绝的测试用例比栈式借用模型少 54%。我们也在 Rocq 中形式化建模了树形借用模型，并用 Simuliris [7] 框架构建了一个关系程序逻辑，证明了树形借用模型优化的正确性。栈式借用模型上生效的几乎所有优化都在树形借用模型上生效；唯一的例外是在原始程序的第一次写入之前插入新写入的优化——而这是为了让树形借用模型与现实世界的代码更兼容所作的权衡之一。据我们所知，树形借用模型支持 Rust 编译器已在运用的别名优化（如例 1 所示）；而不同于栈式借用模型的是，在树形借用模型下能够证明重排序相邻的读取操作是健全的。

本文中的其余部分结构如下：我们首先以示例驱动的方式介绍树形借用模型的核心（第 2 节）。接下来，我们用保护器 *(protector)* 扩展树形借用，这将显著增强可以执行的优化（第 3 节）。之后，我们将新模型与其前身——栈式借用模型——进行详细比较，包括上面提到的实证评估（第 4 节）。最后，我们将展示一些已证明正确的树形借用模型下的优化案例（第 5 节），并以对相关和未来工作的讨论收尾（第 6、7 节）。

[2]这看似是栈式借用模型不允许内联，但实际情况更加微妙。行为上的差异源于表层 Rust 被繁饰 *(elaborate)* 为内部中间表示形式的方式不同；而中间表示形式级别的内联对于栈式借用模型而言是健全的。

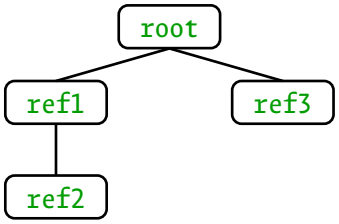
2. 树形借用模型基础

在本节中，我们将解释树形借用模型追踪引用和检测未定义行为背后的基本机制。我们不假设读者事先了解 Rust 或栈式借用模型。

2.1. 记录别名

树形借用模型的核心思想正如其名：引用被组织成一个树形结构。例如，考虑这个例子：

```
let mut root = 42;
let ref1 = &mut root;
let ref2 = &mut *ref1;
let ref3 = &mut root;
```



例 5

例子中有一个局部变量 `root` 和几个或直接或间接地派生自该变量的引用。右边的树状图展示了树形借用模型如何表示这些引用与局部变量之间的关系。每个新创建的引用都与树上的一个新节点关联，并作为一个子节点插入，其父级是用于创建该引用的节点。因此，引用不仅仅是内存中的一个位置；相反，它是由一个位置和一个标识符所构成的二元组来定义的，该标识符确定了其在树中对应的节点。我们将这一标识符称为引用的标签。

访问的效应　每当内存访问发生时，树形借用模型会将这次访问“通知”给所有引用。每个引用——也就是树上的每个节点——记录一个状态机，该状态机定义了引用应对该访问作出何种“反应”：访问要么被批准（可能会改变状态机的状态），要么被拒绝——也就是程序包含未定义行为。

状态机的状态转移由两个因素决定：一是访问操作是读操作还是写操作；二是作出“反应”的引用与用于此次访问的引用之间是何关系。特别地，我们区分*局部访问 (local accesses)* 和*外部访问 (foreign accesses)*。在访问标签 t_{acc} 之后、计算标签 t_{sm} 的状态机转换时，若 t_{acc} “派生自” t_{sm}——即 t_{acc} 是 t_{sm} 自身或其子节点，则称该访问时对 t_{sm} 的局部访问；若 t_{acc} 是 t_{sm} 的父节点或*旁系 (cousin)* 节点，则称该访问为外部访问。例如，在上面的例 5 中，设 t_{sm} = `ref1`，则对 `ref1` 和 `ref2` 的访问就是局部访问，而对 `ref3` 或 `root` 的访问就是外部访问。总的来说，状态机的字母表被定义为 { 局部读 (↓R)，局部写 (↓W)，外部读 (↑R)，外部写 (↑W) }。

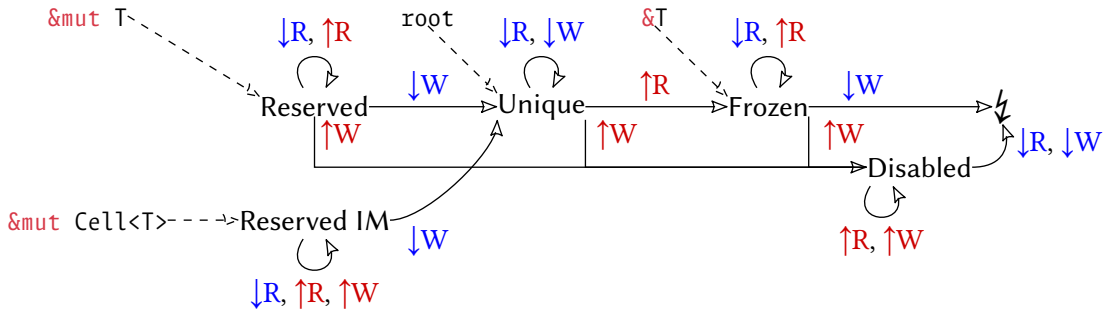


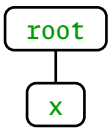
图 1　权限的默认状态机，可变引用入口点由 `&mut T` 标记。抵达状态 ↯ 表明程序包含未定义行为。转移箭头上的标记代表导致该转移的事件：读 (R) 或写 (W)，↑外部或↓局部。

2.2. 可变引用的生存期

现在我们开始逐步定义这个状态机，状态机的完整版本已由图 1 给出。状态机的大部分内容都可通过探讨可变引用来理解。因此，我们从一个简化的可变引用模型开始，随后对其进行扩展，直至该模型能够接受所有安全代码，并允许我们所需的优化。

可变引用的朴素模型　为了从如何处理可变引用中汲取灵感，我们首先将目光投向 Rust 编译器中负责确保引用正确使用的组件——借用检查器（borrow checker）。让我们从一个非常简单的示例开始：

```
let mut root = 42;
let x = &mut root;
*x += 1;
root = 0;
```

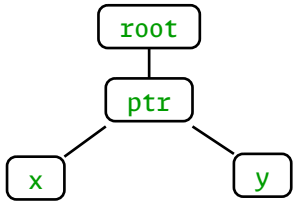


例 6

这里，可变引用 x 由表达式 `&mut root` 创建，故 x 是 `root` 的子级。借用检查器将决定 x 的生存期始于 `&mut root`，且无法在 `root = 0;` 之后延续——那时，父级引用将重新取得所有权，这必然会导致 x 被终止。这一期望的行为可以很容易地用我们的权限和局部/外部访问框架表达：我们使用两个权限 Unique 和 Disabled 表示可变引用的生死。可变引用在一开始被赋予权限 Unique，且只要其权限是 Unique，它就能容许任意的局部访问。而当外部访问发生时，就意味着父级引用收回了指涉物的所有权，故权限应转移至 Disabled，从此刻开始，任何对 x 的局部访问都会触发未定义行为。

尽管这一模型十分简单，但它已经能正确描述可变引用的大部分期望行为。例如，它能拒绝包含“外部写入紧接本地写入”的事件序列，从而恰当地强制执行唯一所有权。为了说明这一点，请回顾例 2：在该示例中，非安全代码为同一位置创建了两个可变引用，并将两者均作为参数传递给一个安全函数。我们发现这一操作违反了可变引用的唯一性原则，因此该代码应被标记为有未定义行为。现在我们再用现在的模型来考察同一段代码（简单起见，例 1 被内联其中），以显明这一模型确能检测未定义行为。

```
let mut root = 42;
let ptr = &mut root as *mut i32;
let (x, y) = unsafe { (&mut *ptr, &mut *ptr) };
*x = 13;
*y = 20;
let val = *x;
```



例 7

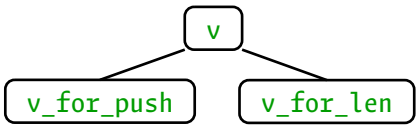
我们记录 x 与 y 的状态：它们皆由 `&mut *ptr` 创建，也就是说它们是从 `root` 间接派生的可变引用。引用 y 是 x 的旁系，因此经由 y 的访问以 x 观之就是一次外部访问，反之亦然。执行 `*x = 13` 会引发一次对 x 的局部写入（使其变为 Unique）和一次 y 的外部写入（使其变为 Disabled）。接着，`*y = 20` 是一次对 Disabled 节点的局部写入，而这是未定义行为，正如预期。

综上所述，我们的模型在此处（以及例 1 中）成功地排除了 x 与 y 之间的别名。尽管对于一个如此简单的模型而言，这一结果颇具前景，但事实将证明仅凭 Unique 和 Disabled 两种属性仍有力不能及之处。接下来，我们将展示前述模型应予完善的两种方式。

二阶段借用和 Reserved 状态　在例 4 中，我们展示了二阶段借用，这正是我们引入树形借用模型的主要动机之一。我们简化的 Unique / Disabled 模型目前对其缺乏支持。

二阶段借用的标准例子是 `v.push(v.len())`，其中 v 具有 `Vec<usize>` 类型 [8]。解糖方法调用记法之后的代码大致是：

```
let v_for_push = &twophase v;
let v_for_len = &v;
let len = Vec::len(v_for_len);
Vec::push(v_for_push, len);
```



例 8

第一行代码计算 push 的第一个参数，接下来的两行代码则计算第二个参数（对 len 的调用也解糖成了一种更显式的形式）。从 v_for_push 的视角看去，在该引用被创建后，当 Vec::len 读取向量长度时，v_for_push 会受到外部读取的影响。而根据前述的状态机，这会将其状态转移至 Disabled，并在之后 Vec::push 时导致未定义行为！因此，这一朴素模型未能考虑到这段应通过的安全代码。

例 8 原本是被借用检查器拒绝的，原因和朴素模型拒绝它的原因基本相同。为克服这一点，Rust 编译器开发者们引入了二阶段借用这一概念，使得某些[3]可变引用的生命以保留阶段开始。在保留阶段，对该引用所指涉的内存的任何读取都是被允许的，即使读取是经由其他引用发生的。在第一次写入可变引用时，该引用随即被激活，它就成为了完全体的可变引用，不再容许别名。我们可以引入一种新的权限来在树形借用模型中对可变引用的保留阶段进行建模，从而将这一行为整合进树形借用模型体系中。我们恰如其分地将这种权限命名为 Reserved。从现在开始，可变引用的生命以 Reserved 而非 Unique 开始。Reserved 状态允许局部和外部读取，因此在整个保留阶段可变引用允许所有读取操作。当引用被激活——也就是它首次经历局部写入之后，它的状态就正式转变为 Unique；和之前一样，当 Unique 经历一次外部访问之后，它就会成为 Disabled。

利用 Frozen 状态启用读-读重排序　只读访问通常被认为是没有副作用的，因此编译器通常可以重排序相邻的只读操作：程序 `let vx = *x; let vy = *y;` 和 `let vy = *y; let vx = *x;` 在任何语境下都应该是等价的[4]。然而在我们的别名模型中，读取操作却不是无副作用的！相反，读取数据的简单操作都会对每个节点所维护的状态机产生效应。例如，一次外部读取就会将 Unique 变为 Disabled。

不幸的是，在当前的状态机草案中，一般而言，相邻的读取不能被重排序。例如，在一个 x 是 root 子级且具有 Unique 状态的语境中（可由 `let mut root = 0; let x = &mut root; *x = 42;` 产生），注意到先读取 x 后读取 root 是可以的，但先读取 root 再读取 x 就会导致未定义行为（经由 root 的读取对 x 而言是外部读取，这会使其变为 Disabled，因而再读取就成了未定义行为）。我们可以将此问题精确定位在由 Unique 向 Disabled 状态的转换过程中：一次外部读取操作（经由 root）导致一个可读引用（即当前处于 Unique 状态的 x）丧失了读取权限（变为 Disabled 状态）。

这表明，在发生外部读取时，Unique 引用不应立即丧失读取能力，仅应失去写入能力。为此，我们引入一个新的权限，称之为 Frozen。这一新权限被作为 Unique 和 Disabled 的中间环节加入，当 Unique 引用遇到外部读取时，即进入此状态；当 Frozen 遇到外部写入时，即离开此状态。Frozen 状态允许任意的局部和外部读取。现在，两种顺序——先读取 root 再读取 x 和先读取 x 后读取 root——会产生相同的状态，因为 x 的状态是 Frozen，且仍容许读取操作。通过引入 Frozen 状态，我们使得读-读重排序重新成为可能。这构成了一个罕见的特例：减少未定义行为反而能带来更多的优化！

处理内存范围　目前为止，我们一直假定每个引用只有一个状态。若仅考虑内存中的单一位置，这一假设便是正确的。为支持多个位置，我们使用类似于 CompCert 的内存模型 [9,10]：内存被组织为一系列连续的分配块 *(allocation)*，每个分配块均由一定数量的字节组成。一个内存位置是一个元组 (a, o)，其中 a 用于标识具体的分配块，o 是块中的偏移量。因此，一个引用由三个部分 $((a, o), t)$ 组成，其中额外包含了标签 t。树形借用机制会为每一个分配块维护一颗标签树。此外，针对每一个内存位置，都存在一个独立的状态机实例；换言之，同一个标签在同一分配块内的不同偏移量处，可能处于不同的状态。当一次内存访问操作涉及多个位置时（例如，一次 4 字节的加载操作会读取 4 个不同的位置），我们会针对该分配块标签树中的所有标签，为每一个受影响的位置独立地更新其对应的状态机。如果任一状态机转入未定义行为状态，则整个内存操作即被判定为未定义行为。

[3]二阶段借用仅对隐式重借用生效，例如，当源码中存在语法 &mut 时，二阶段借用就不会出现。树形借用模型无视了这一区别，并给予所有可变引用一个保留阶段。

[4]这些属于非原子访问，因此即使在并发程序中，读取操作仍可重排。竞态条件则会导致未定义行为。

2.3. 可变引用之外

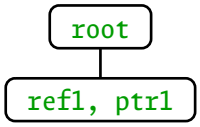
目前为止我们只考虑了可变引用。但 Rust 还有更多类指针类型：共享引用是可变引用的对偶，它们允许别名，但不允许修改。原始指针是 Rust 中的非安全指针构造，不能在安全 Rust 中使用，但允许任意别名。在本节中我们将展示当前的模型能轻松地兼顾这两方面，毋须引入任何新权限。

共享引用　共享引用可与其他共享引用（以及处于保留阶段的可变引用）任意别名，并允许读取访问，但只要它们还处于活跃状态，就不允许写入访问。因此在我们的状态机中，表示共享引用的权限必须允许局部和外部读取，并在尝试局部写入时触发未定义行为。而发生外部写入时，权限应变为 Disabled。看啊：这确是 Frozen 状态所作的！因此，Frozen 除了是可变引用在其生存期即将结束时的临时权限之外，也是共享引用的初始权限。这也给了可变引用从 Unique 到 Frozen 的转移一个新的解释：当一个可变引用不再能维持其唯一性，它就降格成了共享引用。

原始指针　原始指针是 Rust 中最宽松的指针类型，仅在执行引用无法支持的低级操作时使用。指向同一位置的多个原始指针可以同时存在并互为别名，且均可有写入权限。只要原始指针尊重其父级引用的生存期和可变性限制，就很少有它们做不了的事。

在树形借用模型中，原始指针的创建是无操作的：原始指针不会被分配一个新的标签，因此它们必须继承创建它们所用的引用的标签。也就是说，如果从同一个引用创建多个原始指针，经由它们进行的访问可以任意交错——别名模型认为它们都来自“同一来源”，即它们所派生的引用。当引用的生存期结束（即权限变为 Disabled 时），所有原始指针也会失效。

```
let mut root = 42;
let ref1 = &mut root;
let ptr1 = ref1 as *mut i32;
unsafe {
    *ref1 = 43;
    *ptr1 = 44;
    *ref1 = 45;
}
```

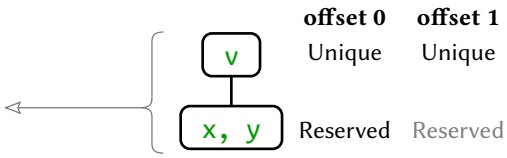


例 9

在这段代码中，以树形借用模型观之，则 `let ptr = ref1 as *mut i32;` 这行代码是无操作的，此后 `ptr1` 也被视同 `ref1`。尽管最后三个赋值操作是在给三个语法上不同的变量赋值，树形借用模型在运行时只认标签，而三次访问的标签全都相同。因此树形借用模型接受此代码，正如预期：因为原始指针允许别名。

树形借用模型甚至允许原始指针访问其父级引用的类型决定的内存范围之外的内存，如下面的例子所示：

```
let mut v = [0u8, 0];
let x = &mut v[0];
let y = (x as *mut i32).add(1);
unsafe { *y = 1; }
```



例 10

在这个例子中，`x` 是指向 `v` 第一个元素的可变引用。使用原始指针转换和指针算术，我们创建出了 `y`，它与 `x` 有着相同的标签，但指向 `v` 的第二个元素。特别地，`y` 指向一个它所派生的引用 `x` 的边界之外的位置！这一模式在低层代码中并不少见。特别地，这在实现一种特设动态多态 *(ad-hoc dynamic polymorphism)* 时非常有用，在这种模式中，某个块前缀处的数据提供了有关其后面的数据的形状的信息。这些信息超出了静态类型可以表达的范围，因此必须使用原始指针来访问后面的数据。

树形借用模型支持这样的用法——例 10 能被这一模型接受。其工作原理是，`&mut v[0]` 为当前分配块中——而不仅仅是在 `v[0]` 类型指示的内存范围中——的每个位置的新引用创建一个权限和与之相关联的状态机。也就是说偏移 1 处的权限——因其超出了类型指示的范围故用灰色显示——也是 Reserved。这就使得用不安全代码动态地将引用扩展到更大的范围成为可能。与此同时，这一模型（使用前面已经介绍过的机制）仍能确保派生自不同可变引用的原始指针只能被用于改变内存中不相交的部分。

2.4. 内部可变性

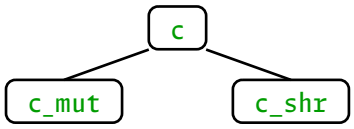
Rust 的引用会阻止对存在别名的状态进行修改，但有时共享可变状态确是必要的。Rust 通过内部可变性 *(interior mutability)* 机制来支持这一点，该机制允许使用内部由非安全代码实现的库类型，来暴露经过严格控制的共享可变状态。具有内部可变性的类型不遵循 Rust 通常的别名原则，因此必须使用特殊的 `UnsafeCell<T>` 类型作为标记，表明类型为 `T` 的内部数据可经由共享引用修改。

在别名方面，指向内部可变类型的共享引用本质上等同于原始指针；因此，树形借用模型对它们的建模方式也如出一辙：共享引用会继承其父级引用的标签。这便使得对这些引用的处理变得微不足道：和原始指针一样，它们可与派生自同一父级的其他原始指针自由地发生别名。

指向内部可变类型的可变引用在变为 Unique 之后就和常规的可变引用没有区别了。然而，在可变引用的保留阶段，可能存在指向同一位置的共享引用。而因为经由共享引用也能修改内部可变类型的数据，故处于保留阶段的可变引用不止要能容许外部读取，还要容许外部写入。例 11 给出了这样一个例子——和例 8 类似，但 `c` 的类型是 `Cell<i32>`，它具有内部可变性。因此，`Cell::replace` 可向单元中写入，而这一写入决不能令 `c_mut` 变为 Disabled。

```rust
fn foo(c : &mut Cell<i32>, n : i32) {
    *c.get_mut() = n;
}

let mut c = Cell::new(1);
let c_mut = &twophase c;
let c_shr = &c;
let val = Cell::replace(c_shr, 42); // 写入
foo(c_mut, val);
```



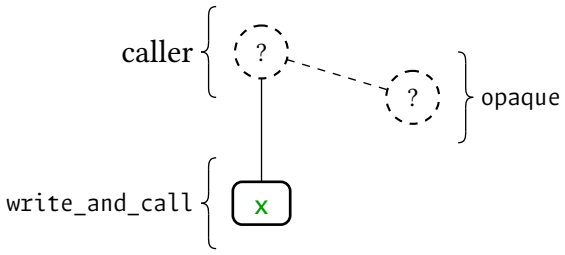
例 8

为建模这一行为，我们引入一个新的权限 Reserved IM，它在其他方面和 Reserved 均相同，但它允许外部写入。Reserved IM 是指向内部可变类型的可变引用的新状态机入口点。

3. 树形借用模型进阶：确保引用存活

在上一节中，我们构建了一个别名模型，该模型能接受所有安全的 Rust 代码，同时又能排除部分我们认定存在问题的非安全代码。不幸的是，就其所能实现的优化而言，这一别名模型相当薄弱：Rust 编译器目前已在执行的一些优化并不能通过前述模型加以佐证。特别地，编译器假设作为函数实参传入的引用在整个函数调用期间都是唯一 / 不可变的，并用这一假设来优化以下示例：

```rust
fn write_and_call(x: &mut i32,
                  opaque: impl Fn()) {
    *x = 13; // 变换：消除这次写入
    opaque();
    *x = 20;
}
```

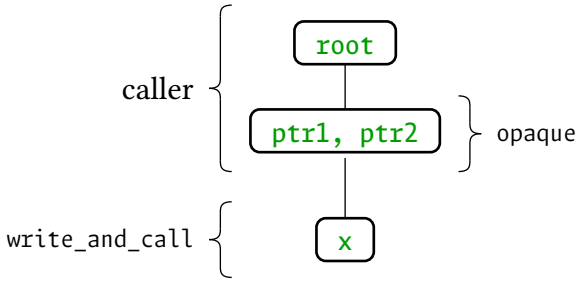


例 9

这里，opaque 是一个不接受任何实参、也不返回任何东西的闭包。我们的目标是消除第一次写入。注意到，这是一项相当强力的优化：x 指向以未知方式分配的内存，而 opaque 也是任意的未知代码——在这种情境下通常根本无法执行任何别名优化！

直觉上，树形借用模型应该能让这一优化成为可能，因为 opaque 根本不可能访问 x 所指向的内存。无论如何，若要访问 x 所指向的内存，opaque 就必须使用某些相对于 x 外部的别名（如代码右侧图示所示）。这确是未定义行为，因为它违反了 x 的唯一性：这一访问会使得 x 从 Unique 变为 Disabled，故 `*x = 20` 是未定义行为。然而问题在于，函数 opaque 可能永不返回，在这种情况下 `*x = 20` 永远不会被执行，且不会导致未定义行为。这一点由以下语境所佐证：

```rust
let mut root = 42;
let ptr1 = &mut root as *mut i32;
let opaque = move || {
    let ptr2 = ptr1;
    println!("{}", unsafe { *ptr2 });
    std::process::exit(0); // 退出程序
};
write_and_call(unsafe { &mut *ptr1 },
               opaque);
```



参考文献

1. Jung R, Dang H-H, Kang J, Dreyer D. Stacked Borrows: An Aliasing Model for Rust. Proc ACM Program Lang. 2020;4: 41:1–41:32. doi:10.1145/3371109

2. Unsafe Code Guidelines. Issue #134: Stacked Borrows: raw pointer usable only for T too strict?. 2019. Available: https://github.com/rust-lang/unsafe-code-guidelines/issues/134

3. Unsafe Code Guidelines. Issue #248: Stacked Borrows: track raw pointers more precisely. 2020. Available: https://github.com/rust-lang/unsafe-code-guidelines/issues/248

4. Miri. Issue #3082: Is a stacked borrows error in iced-x86 a problem?. 2023. Available: https://github.com/rust-lang/miri/issues/3082

5. Miri. Issue #3657: Possible false positive of stacked borrow rules. 2024. Available: https://github.com/rust-lang/miri/issues/3657

6. Fetisov A. Aliasing of raw pointers. 2024. Available: https://internals.rust-lang.org/t/aliasing-of-raw-pointers/21746

7. Gäher L, Sammler M, Spies S, Jung R, Dang H-H, Krebbers R, et al. Simuliris: A Separation Logic Framework for Verifying Concurrent Program Optimizations. Proc ACM Program Lang. 2022;6: 1–31. doi:10.1145/3498689

8. The Rust Team. Rust Compiler Development Guide: Two-Phase Borrows. 2020. Available: https://rustc-dev-guide.rust-lang.org/borrow_check/two_phase_borrows.html

9. Leroy X, Appel A, Blazy S, Stewart G. The CompCert Memory Model, Version 2. 2012. Report No.: RR-7987. Available: https://hal.inria.fr/hal-00703441

10. Leroy X, Blazy S. Formal Verification of a C-like Memory Model and Its Uses for Verifying Program Transformations. J Autom Reason. 2008;41: 1–31. doi:10.1007/s10817-008-9099-0

11. Unsafe Code Guidelines. Issue #256: Storing an object as &Header, but reading the data past the end of the header. 2020. Available: https://github.com/rust-lang/unsafe-code-guidelines/issues/256

12. Unsafe Code Guidelines. Issue #273: Stacked Borrows: using the topmost ``untagged'' item is not ``monotonic''. 2021. Available: https://github.com/rust-lang/unsafe-code-guidelines/issues/273

13. Beck C, Yoon I, Chen H, Zakowski Y, Zdancewic S. A Two-Phase Infinite/Finite Low-Level Memory Model: Reconciling Integer–Pointer Casts, Finite Space, and undef at the LLVM IR Level of Abstraction. Proc ACM Program Lang. 2024;8: 1–29. doi:10.1145/3674652

14. Hathhorn C, Ellison C, Rosu G. Defining the Undefinedness of C. In Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). Portland, OR, USA; 2015. pp. 336–345. doi:10.1145/2737924.2737979

15. ISO. C17 Standard. 2017.

16. Jung R, Krebbers R, Jourdan J-H, Bizjak A, Birkedal L, Dreyer D. Iris from the Ground Up: A Modular Foundation for Higher-Order Concurrent Separation Logic. J Funct Program. 2018;28: e20. doi:10.1017/S0956796818000151

17. Kang J, Hur C-K, Mansky W, Garbuzov D, Zdancewic S, Vafeiadis V. A Formal C Memory Model Supporting Integer-Pointer Casts. In Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). 2015. pp. 326–335. doi:10.1145/2737924.2738005

18. Krebbers R. Aliasing Restrictions of C11 Formalized in Coq. In Certified Programs and Proofs – Third International Conference (CPP 2013). Melbourne, VIC, Australia; 2013. pp. 50–65. doi:10.1007/978-3-319-03545-1_4

19. Krebbers R. The C Standard Formalized in Coq. Doctoral dissertation. 2015. Available: https://robbertkrebbers.nl/thesis.html

20. Krebbers R, Jourdan J-H, Jung R, Tassarotti J, Kaiser J-O, Timany A, et al. MoSeL: A General, Extensible Modal Framework for Interactive Proofs in Separation Logic. Proc ACM Program Lang. 2018;2: 77:1–77:30. doi:10.1145/3236772

21. Lee J, Hur C-K, Jung R, Liu Z, Regehr J, Lopes NP. Reconciling High-Level Optimizations and Low-Level Code in LLVM. Proc ACM Program Lang. 2018;2: 125:1–125:28. doi:10.1145/3276495

22. Lopes NP, Lee J, Hur C-K, Liu Z, Regehr J. Alive2: Bounded Translation Validation for LLVM. In Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI). 2021. pp. 65–79.

23. Louwrink D. A Separation Logic for Stacked Borrows. Master's thesis. 2021. Available: https://eprints.illc.uva.nl/id/eprint/1790/

24. McCormack I, Sunshine J, Aldrich J. A Study of Undefined Behavior Across Foreign Function Boundaries in Rust Libraries. CoRR. 2024. Available: https://arxiv.org/abs/2404.11671

25. Memarian K, Gomes VBF, Davis B, Kell S, Richardson A, Watson RNM, et al. Exploring C Semantics and Pointer Provenance. Proc ACM Program Lang. 2019;3: 67:1–67:32. doi:10.1145/3290380

26. Memarian K, Matthiesen J, Lingard J, Nienhuis K, Chisnall D, Watson RNM, et al. Into the Depths of C: Elaborating the De Facto Standards. In Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). 2016. pp. 1–15. doi:10.1145/2908080.2908081

27. Memarian K, Sewell P. N2014: What is C in Practice? (Cerberus Survey v2): Analysis of Responses. 2016. Available: http://www.cl.cam.ac.uk/~pes20/cerberus/notes50-survey-discussion.html

28. Norrish M. C Formalised in HOL. Doctoral dissertation. 1998.

29. LLVM Project. LLVM Language Reference. 2024. Available: https://llvm.org/docs/LangRef.html

30. The Coq Team. The Coq Proof Assistant. 2023. Available: https://coq.inria.fr/

31. Villani N, Hostert J, Dreyer D, Jung R. Tree Borrows – Artifact. 2025. doi:10.5281/zenodo.15002703

32. Zakowski Y, Beck C, Yoon I, Zaichuk I, Zaliva V, Zdancewic S. Modular, Compositional, and Executable Formal Semantics for LLVM IR. Proc ACM Program Lang. 2021;5: 1–30. doi:10.1145/3473572